TP2 – Robotique : Resoudre un labyrinthe avec Astar

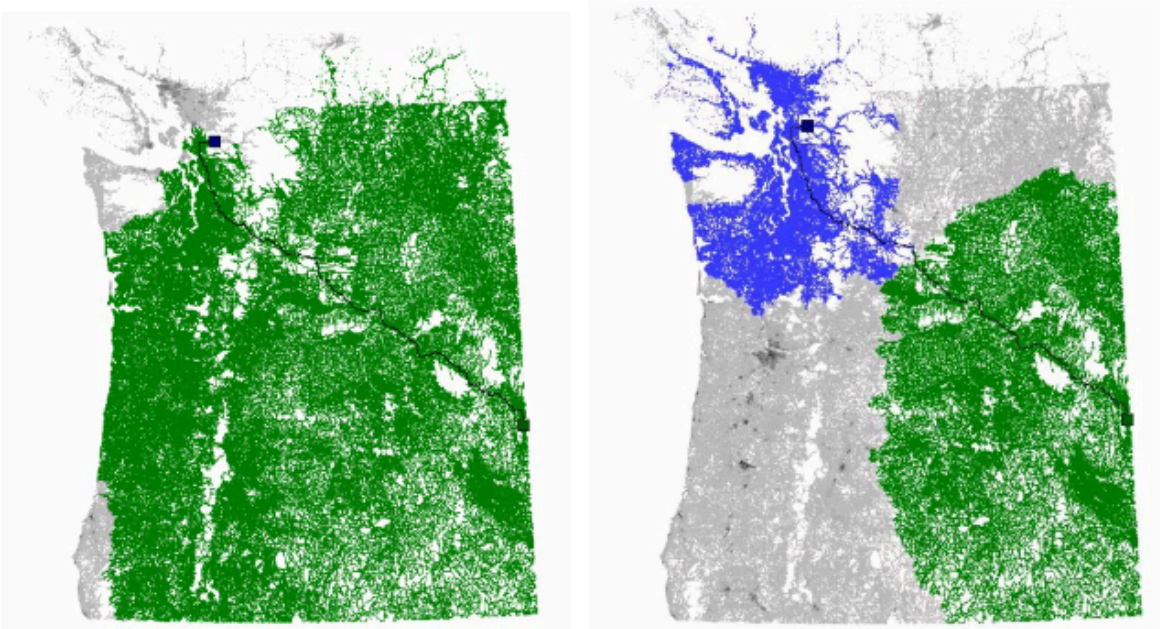
Polytech Paris-Saclay APP5 EIE 19/03/2026

Realise par: CHIHA Faress, DAGGAG Anas

Objectifs pedagogiques

Implementer et analyser la recherche bidirectionnelle pour :

- Dijkstra classique vs Dijkstra bidirectionnel
- Astar classique vs Astar bidirectionnel
- mesurer temps d'execution et nombre de sommets explores



(a) Dijkstra classique. (b) Dijkstra bidirectionnel.

FIGURE 1 – Illustration de l'algorithme Dijkstra classique et bidirectionnel (source : Goldberg et al.).

Le TP2 reutilise du code TP1 :

- `Maze.py` : version unidirectionnelle (Astar et Dijkstra)
- `main.py` : generation de labyrinthes
- `bidirectional.py` : nouvelle classe `BiDirectionalMaze`
- `tests_bidirectional.py` : benchmarks TP2

Adaptations faites :

- correction de reconstruction du chemin bidirectionnel (pas de doublon du point de rencontre)
- garde anti-boucle lors de la reconstruction
- scenarios de test TP2 sans bonus (cout uniforme), pour une comparaison correcte

Principe du Dijkstra bidirectionnel

Deux recherches sont lancees en parallele :

- avant : `start -> goal`
- arriere : `goal -> start`

On maintient le meilleur cout de rencontre `best_cost`. Critere d'arret utilise :

$$\text{minF_avant} + \text{minF_arriere} \geq \text{best_cost}$$

Ce critere signifie qu'aucun chemin meilleur ne peut etre trouve.

Justification courte (correction du critere) :

- `minF_avant` est une borne inferieure du cout restant cote avant.
- `minF_arriere` est une borne inferieure du cout restant cote arriere.
- Donc `minF_avant + minF_arriere` est une borne inferieure de tout nouveau chemin complet.
- Si cette borne est deja superieure ou egale au meilleur chemin rencontre `best_cost`, aucune amelioration n'est possible.
- On peut donc arreter sans perdre l'optimalite.

Travaux et Questions

Illustrations du pathfinding en action

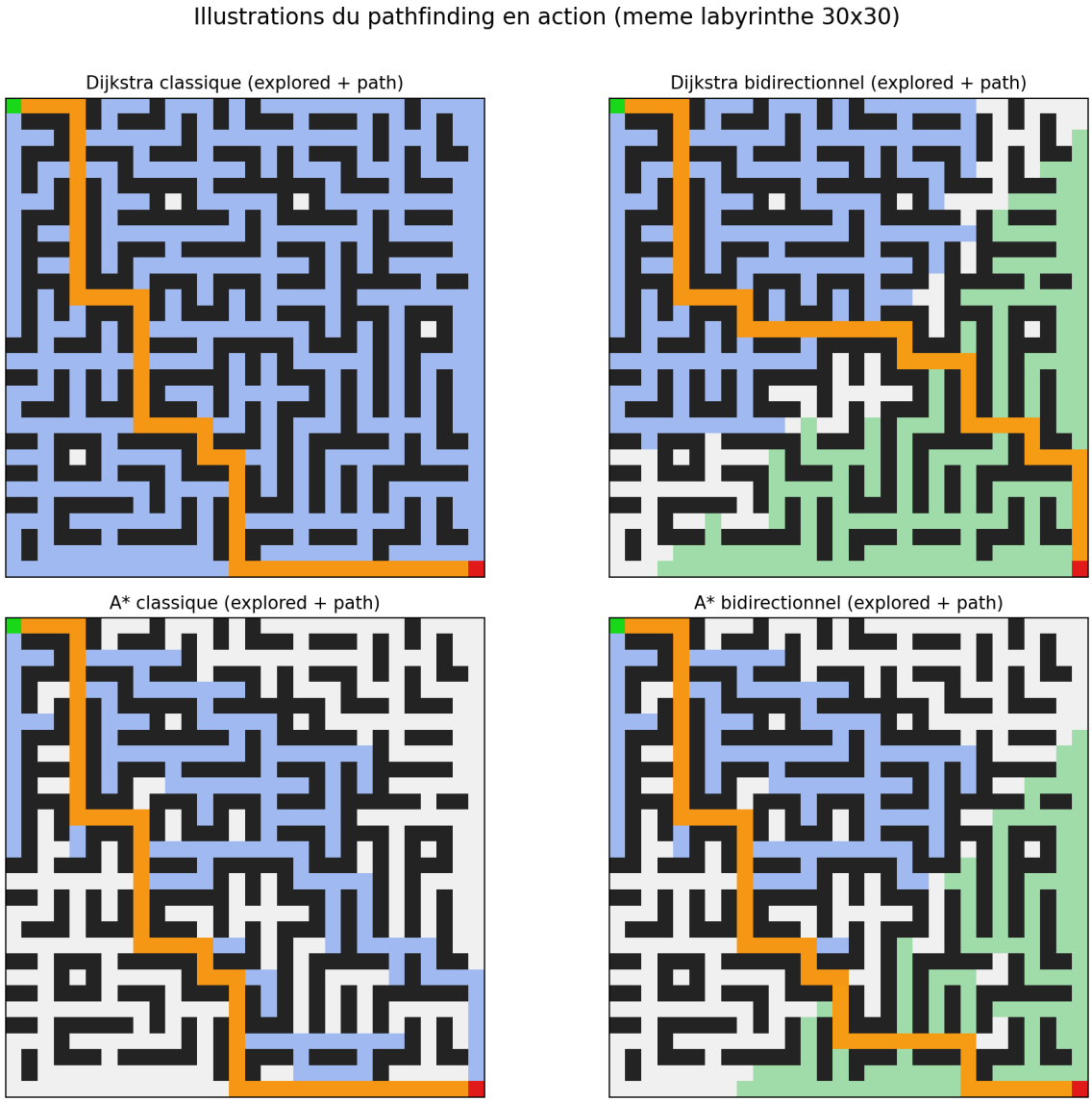


Figure 1: Illustrations de Dijkstra et Astar en version classique et bidirectionnelle sur le meme labyrinthe 30x30. Couleurs: obstacle = noir, exploration avant = bleu, exploration arriere = vert, chemin final = orange, start = vert vif, goal = rouge.

Lecture rapide:

- Dijkstra classique explore le plus largement.
- Dijkstra bidirectionnel montre deux fronts de recherche qui se rejoignent.
- Astar classique reste tres focalise vers le but.
- Astar bidirectionnel montre aussi deux fronts, mais n'est pas toujours plus efficace en exploration.

1) Dijkstra bidirectionnel : implementation + critere d'arret

Enonce (Q1) : Implementer l'algorithme de Dijkstra bidirectionnel. Quel est le critere d'arret ? Le tester sur le petit exemple.

Reponse :

- implementation dans `bidirectional.py` (`dijkstra_bidirectional`)

- critere d'arret :

$$\text{minF_avant} + \text{minF_arriere} \geq \text{best_cost}$$

- test 10x10 sans obstacle : chemin trouve de longueur 19

Verdict Q1 : Critere d'arret implemente et valide sur petit cas.

2) Tests sur des labyrinthes plus grands

Enonce (Q2) : Tester Dijkstra bidirectionnel sur des labyrinthes de plus grande taille.

Resultats experimentaux (`tests_bidirectional.py`) :

Cas	Dijkstra uni (explores)	Dijkstra bi (explores)	Reduction	Temps bi
10x10 sans obstacle	99	92	7.1%	0.59 ms
10x10 obstacles 0.2	83	70	15.7%	-
12x8 murs verticaux	80	76	5.0%	-
30x30 maze_pattern	520	438	15.8%	1.28 ms

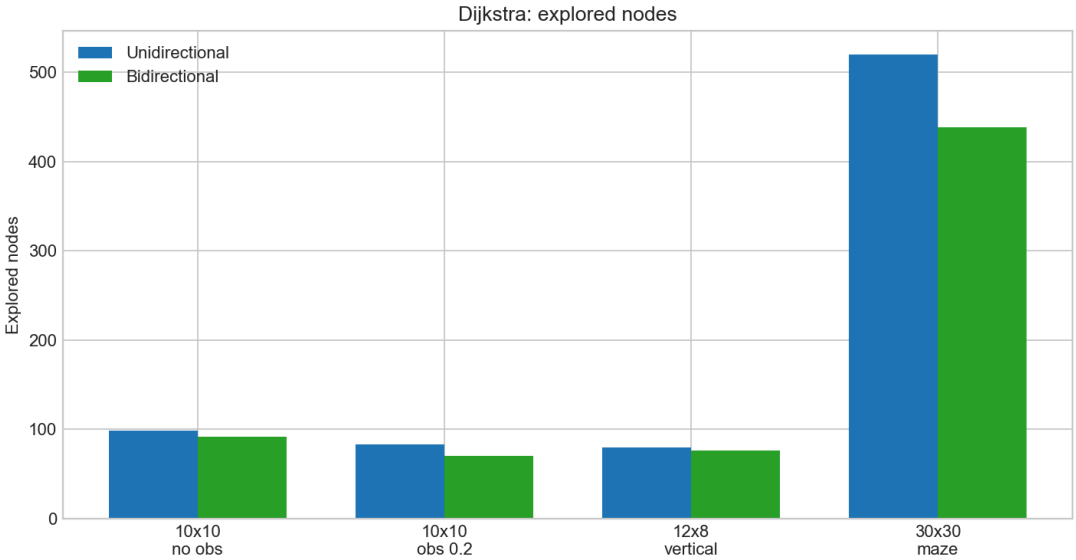


Figure 2: Comparaison du nombre de sommets explores par Dijkstra unidirectionnel et bidirectionnel.

Conclusion partielle : Dijkstra bidirectionnel explore moins de sommets que Dijkstra classique sur tous les cas testes.

Verdict Q2 : Sur nos tests, Dijkstra bidirectionnel est systematiquement plus sobre en exploration.

3) Comparaison avec NetworkX `bidirectional_dijkstra`

Enonce (Q3) : Comparer votre algorithme avec l'algorithme integre `bidirectional_dijkstra` de NetworkX.

Benchmark realise avec `benchmark_networkx_tp2.py` sur 30x30 `maze_pattern` :

Algorithme	Longueur	Cout	Temps	Observation
Notre Dijkstra bidirectionnel	59	58.00	1.50 ms	explores = 434
NetworkX bidirectional_dijkstra	59	58.00	1.25 ms	meme chemin optimal

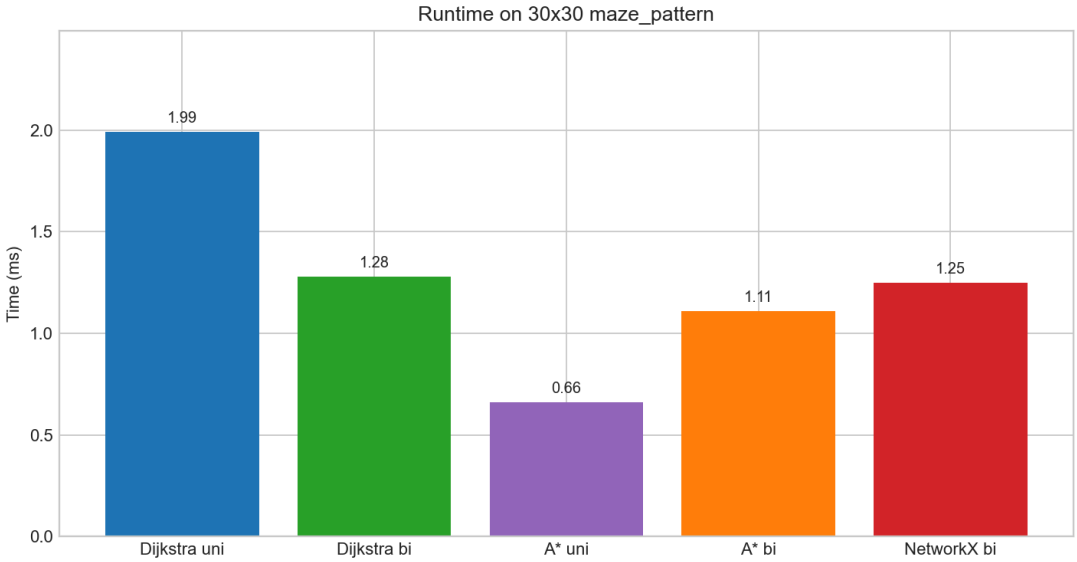


Figure 3: Comparaison des temps d'execution sur le cas 30x30 (nos implementations + NetworkX).

Les deux implementations trouvent le meme resultat optimal. NetworkX est legerement plus rapide sur ce test.

Verdict Q3 : Notre implementation est correcte (meme chemin/cout), avec un leger ecart de performance face a NetworkX.

4) Dijkstra classique vs Dijkstra bidirectionnel

Enonce (Q4) : Quel algorithme explore le moins de sommets ?

Reponse : Dijkstra bidirectionnel explore moins de sommets dans tous les tests realises (gain de 5% a 15.8%).

Verdict Q4 : Dijkstra bidirectionnel gagne sur le critere "sommets explores".

5) Implementation de Astar bidirectionnel

Enonce (Q5) : Implementer l'algorithme Astar bidirectionnel.

Reponse : implementation dans `bidirectional.py` (`astar_bidirectional`) avec heuristique de Manhattan dans les deux directions.

Verdict Q5 : Astar bidirectionnel implemente et fonctionnel.

6) Comparaison Dijkstra bidirectionnel vs Astar bidirectionnel

Enonce (Q6) : Comparer Dijkstra bidirectionnel et Astar bidirectionnel. Commenter.

Resultats (`tests_bidirectional.py`) :

Cas	Astar uni explores	Astar bi explores	Variation	Temps Astar uni	Temps Astar bi
10x10 sans obstacle	99	92	7.1% de reduction		0.44 ms
10x10 obstacles 0.2	47	48	Astar bi legerement pire		-

Cas	Astar uni explores	Astar bi explores	Variation	Temps Astar uni	Temps Astar bi
12x8 murs verticaux	80	70	12.5% de reduction		-
30x30 maze_pattern	160	268	Astar bi nettement pire	0.66 ms	1.11 ms

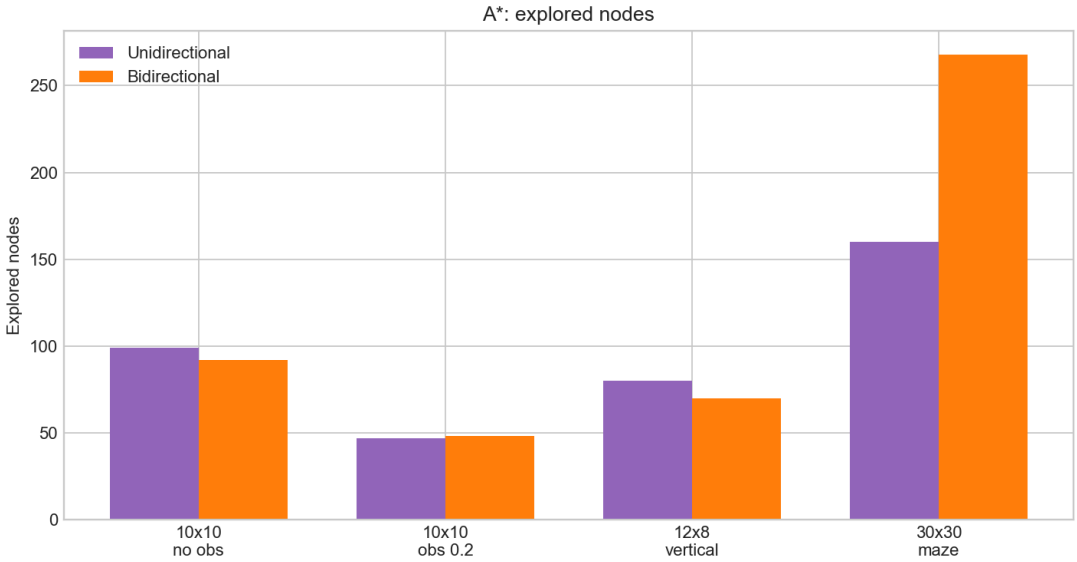


Figure 4: Comparaison du nombre de sommets explores par Astar unidirectionnel et bidirectionnel.

Commentaires :

- contrairement a Dijkstra, Astar bidirectionnel n'est pas toujours meilleur que Astar unidirectionnel
- sur les grands labyrinthes testes ici, Astar unidirectionnel reste le plus performant
- la version bidirectionnelle de Astar demande un critere de convergence tres soigne pour etre avantageuse

Verdict Q6 : Astar bidirectionnel ne garantit pas un gain pratique sur nos jeux de tests.

Limites et validite experimentale

Points de validite :

- Tous les algorithmes sont compares sur les memes labyrinthes.
- Les couts sont uniformises (sans bonus) pour eviter un biais.
- Les conclusions principales sont stables sur plusieurs tailles de grille.

Limites :

- Les mesures de temps sont locales a une machine et a une seule implementation.
- Le nombre de repetitions temporelles reste limite.
- Astar bidirectionnel depend fortement du critere de convergence et de l'heuristique choisie.

Ameliorations possibles :

- moyenner les temps sur 30 a 100 executions par cas,
- ajouter des labyrinthes aleatoires supplementaires,
- comparer d'autres heuristiques et variantes de terminaison bidirectionnelle.

Conclusion

Le TP2 est implemente en reutilisant la base du TP1, avec adaptation au bidirectionnel.

Resultat principal :

- Dijkstra bidirectionnel apporte un gain regulier en exploration et en temps
- Astar bidirectionnel n'apporte pas de gain systematique sur nos cas de test

En pratique, pour cette implementation et ces labyrinthes :

- meilleur compromis stable : Dijkstra bidirectionnel
- meilleur temps brut observe : Astar unidirectionnel

Fichiers utilises

- `Maze.py`
- `main.py`
- `bidirectional.py`
- `tests_bidirectional.py`
- `benchmark_networkx_tp2.py`